

Rapport Projet Web

1. Mise en place de la structure du projet

Le projet a été conçu en respectant l'architecture recommandée par le framework Django, afin de garantir une organisation claire et évolutive. Deux applications principales ont été créées. L'application `accounts` est dédiée à la gestion des utilisateurs, notamment l'inscription, la connexion et la déconnexion. L'application `chats` est quant à elle responsable du système de messagerie et de la gestion des salons de discussion.

Par ailleurs, les répertoires `templates/` et `static/` ont été ajoutés pour structurer correctement les fichiers HTML, CSS et JavaScript. Cette organisation permet de séparer la logique applicative de la présentation et prépare le projet à de futures évolutions fonctionnelles.

La séparation du projet en plusieurs applications améliore la modularité et facilite la maintenance du code. Le respect de l'architecture MVT (Modèle–Vue–Template) de Django renforce la lisibilité du projet et assure une meilleure cohérence globale. Enfin, une organisation claire dès le départ limite les conflits en cas de travail collaboratif et simplifie l'ajout de nouvelles fonctionnalités.

2. Implémentation de la gestion des utilisateurs (application `accounts`)

L'application `accounts` repose sur les mécanismes d'authentification intégrés à Django. Elle permet de gérer l'inscription des nouveaux utilisateurs, leur connexion ainsi que leur déconnexion. Ces fonctionnalités utilisent les vues fournies par Django et son système de gestion des sessions, garantissant une intégration fiable et robuste.

Les outils d'authentification natifs de Django offrent un haut niveau de sécurité, en particulier pour la gestion et le stockage des mots de passe. Ne pas réimplémenter un système d'authentification permet de réduire les risques d'erreurs et d'assurer le respect des bonnes pratiques du framework. Cette approche permet également de concentrer les efforts de développement sur les fonctionnalités spécifiques au système de chat.

3. Structuration de l'application de messagerie (application `chats`)

L'application `chats` a été structurée pour accueillir les différentes fonctionnalités du système de discussion. Un modèle `Message` a été mis en place afin de gérer le contenu textuel des messages, leur association à un auteur et, à terme, leur lien avec un salon de discussion. Cette structure constitue la base de la persistance des échanges entre utilisateurs.

De plus, la page de chat a été préparée dans les templates afin d'être compatible avec les fonctionnalités dynamiques prévues, telles que l'actualisation automatique des messages, l'utilisation d'AJAX ou encore la gestion des emojis.

Le modèle Message est un élément fondamental pour toute application de messagerie. Préparer l'interface dès le début du projet facilite l'intégration ultérieure des mécanismes côté client (JavaScript ou jQuery) demandés dans le cahier des charges. Cette organisation rend également plus simple l'ajout de fonctionnalités futures, comme la modération, la gestion des droits utilisateurs ou la création de salons multiples.

4. Mise en place du salon de discussion et des échanges dynamiques

Un système de salon de discussion a été mis en place afin de permettre aux utilisateurs d'échanger des messages en temps réel. L'envoi des messages a été optimisé grâce à l'implémentation de la technologie AJAX, ce qui permet d'éviter le rechargement complet de la page lors de l'envoi d'un message. Ainsi, lorsque l'utilisateur envoie un message, celui-ci apparaît instantanément dans l'interface, offrant une expérience plus fluide et réactive.

Un mécanisme d'actualisation automatique (auto-refresh) a également été ajouté. Celui-ci interroge régulièrement le serveur afin de vérifier la présence de nouveaux messages. Pour cela, une API dédiée a été développée afin de ne récupérer que les messages récents, sans recharger l'ensemble du contenu. Cette approche permet de limiter la quantité de données échangées et d'améliorer les performances de l'application.

L'utilisation d'AJAX et d'une API légère améliore considérablement l'expérience utilisateur en rendant la discussion plus dynamique et interactive. Cette solution offre un fonctionnement proche des applications de messagerie modernes, tout en optimisant les performances du site. Elle permet également de garantir une meilleure scalabilité et une intégration plus simple de fonctionnalités futures, telles que la notification en temps réel ou l'affichage des statuts de connexion.

5. Permissions et fonctionnalités admin:

I - Bannir des utilisateurs

Nous avons créé un bouton statique pour cette fonctionnalité. Bien que nous aurions pu utiliser un formulaire classique comme pour les autres actions, cette approche nous permet de démontrer tout ce qui a été appris en TP.

Les clics sur ces boutons sont écoutés dans le fichier action.js via un listener global, qui redirige vers la fonction correspondante. Cette approche est nécessaire car il est impossible de savoir à l'avance quels boutons seront chargés sur la page (les utilisateurs peuvent arriver dynamiquement).

La fonction `ban_user` dans `views.py` crée un objet `Ban` qui sert ensuite à filtrer l'affichage des pages pour chaque utilisateur lors du chargement de `home.html`. Chaque action critique est sécurisée : côté front-end par un `confirm()`, et côté back-end par la vérification des droits de l'utilisateur.

II - Supprimer des salons:

Cette fonctionnalité utilise un bouton statique unique, relié à un listener direct. La fonction `delete_salon` supprime l'objet correspondant en utilisant la méthode `delete()`. Deux couches de

sécurité sont mises en place : l'utilisateur doit confirmer l'action et doit être le créateur du salon pour que la suppression soit autorisée.

III - Supprimer des messages:

Pour la suppression de messages, nous utilisons un mélange des deux approches précédentes. Les clics sont écoutés via un listener global, qui redirige vers la fonction `delete_message`. L'objet message correspondant est ensuite supprimé avec la méthode `delete()`. La vérification des droits est également appliquée : seul le créateur du salon peut supprimer les messages.

6. Hébergement:

Suivant les instructions du TP5, le projet a été déployé sur `Alwaysdata.net` via une connexion SSH. Le type de projet a été configuré en Python WSGI, le chemin vers le projet précisé, et le domaine ajouté aux `ALLOWED_HOSTS` dans `settings.py`. Le projet est désormais accessible en ligne via le domaine configuré.